Use the Citation Checker in EduGenAI

Register this tool as an EduGenAI Extension so the assistant can verify a paper's references against OpenAlex — inside your own EduGenAI chat, with no separate LLM key.

How it works

An EduGenAI Extension is a function-calling tool: you give EduGenAI an HTTP endpoint and a function definition, and its assistant calls that endpoint as a JSON request whenever it needs to. The work splits across two sides:

- **EduGenAI's assistant** reads the paper and extracts the reference list plus the sentences that cite each source.
- **This extension endpoint** looks each reference up in OpenAlex and checks title, authors, year, journal, DOI, volume, issue and pages — deterministically, with no LLM — and returns the abstract.
- **EduGenAI's assistant** compares the citing sentence to that abstract and flags misquotes.

Because the reasoning stays on EduGenAI's side, the extension needs no LLM API key. It only wraps OpenAlex, which is free. An optional OpenAlex Premium key can be stored securely in the Headers / Azure Key Vault section (Step 3).

Before you start

Deploy this app at a public HTTPS URL — EduGenAI calls it server-to-server. Everywhere below, replace `https://YOUR-DEPLOYMENT-HOST` with your deployment's address.

Step 1 — Open the Extension builder

In EduGenAI, create a new Extension (Name, Short description, Headers, Functions).

Step 2 — Name and describe it

Name

```
OpenAlex Citation Verifier
```

Short description

```
Verifies a paper's references against OpenAlex: checks that each
work exists and that the printed title, authors, year, journal,
DOI and pages match, and returns the abstract. Run it whenever
the user says "start", "start the extension", "check", or asks
to verify citations or references.
```

Detail description (paste verbatim — this is the assistant's instruction)

```
Use this extension to fact-check the reference list of an uploaded paper.
(Emoji are written as words below; copy the exact version with emoji from the
/edugenai page.)

TRIGGER - WHEN TO RUN. Run all four steps below end to end, WITHOUT asking any
clarifying question first, whenever the user gives ANY go-ahead while a document
is present in the conversation. Triggers include (not exhaustive): "start",
"go", "run", "check", "verify", "start the check", "start the extension", "run
the extension", "start the references extension", "start the reference
verification", "check my citations", "verify my references", any message
containing "start" or "run" together with "check", "extension" or "references",
or a paper uploaded with no other instruction. Never respond to a trigger with a
greeting, a menu of options, or a request to paste the bibliography - the
document already in the conversation IS the input. Only if NO document has been
provided at all, ask for one, then run.
The steps run IN ORDER: you must finish STEP 1 (reading the document and
```

extracting the references yourself) BEFORE calling the function in STEP 2. Never call verify_references with an empty, placeholder, or not-yet-extracted references array.

AUDIENCE. The user is a marker / examiner / supervisor checking SOMEONE ELSE'S paper (typically a student's). Never address the user as the paper's author: write "the paper" or "the student's paper", never "your paper" / "you cite". Report findings as facts for a reviewer. Never give feedback or improvement advice to the writer.

OUTPUT DISCIPLINE. Produce ONLY the output defined in Step 4 - the table, the Details section, the Missing-references section when relevant, and the one-line summary. NOTHING else: no APA or formatting critique, no style fixes, no suggested extra literature, no praise, no summary of what the paper is about, no advice - unless the user explicitly asks for such things in a later message. EXCEPTION - these are citation ERRORS, not style, and MUST be reported: wrong author names, wrong AUTHOR ORDER (e.g. the text or reference says "Kahneman & Tversky, 1974" where the record is Tversky & Kahneman), wrong year/journal/DOI/pages, in-text citations that contradict their reference entry, and citations missing from the reference list. Punctuation, capitalisation, and italics are style - stay silent on those.
If the function call fails or returns an empty result, say exactly that in one or two sentences ("the verification service returned no data, so the references could not be checked - try again") and STOP: do not substitute an APA review, a plausibility opinion, or any other unrequested analysis.

STEP 1 - EXTRACT. Read the document. For every entry in the reference list, pull out: title, the full list of author names in order, whether it ends in "et al.", year, DOI (only if printed), journal/venue, volume, issue, and page range. Also find the sentence(s) in the body where each source is cited, with one sentence of context on each side. Keep those citation sentences in your own working notes for Step 3 - do NOT send them to the function.
Also list every IN-TEXT citation (e.g. "(Smith, 2020)", "Jones et al. (2019)") that has NO matching entry in the reference list - these ORPHAN CITATIONS cannot be looked up by any reader. Keep them in your notes for Step 4; do not send them to the function. If the paper has NO reference list at all, every in-text citation is an orphan: do not fabricate entries and do not send author+year-only items - skip the function call and go straight to Step 4, reporting every citation as missing its reference.

STEP 2 - VERIFY. Call the "verify_references" function once, passing every reference that has at least a title or a DOI (an entry with neither cannot be identified and comes back as lookup_failed - report it as unverifiable, never as a hallucination). For each it returns: "badge" and "severity" (0-100), "status" (found / fuzzy / not_found / lookup_failed), "mismatched_fields" and "minor_fields", a "field_check" comparing each printed detail to OpenAlex (reference_value vs openalex_value; each field's status is "match", "close" = a minor naming variation such as an abbreviated journal name, or "mismatch"), the matched "work" (authors, year, venue, doi, url) and its "abstract", and - for FUZZY matches only, where "work" is null and "field_check" is empty - a "candidates" array of the closest works (each with title, authors, year, venue, doi, url).

STEP 3 - JUDGE MISQUOTES (THE CORE OF THIS TOOL - NEVER SKIP). This comparison is the main reason the user runs the check. For EVERY reference whose status is "found" and whose returned "abstract" is non-empty, you MUST compare the citation sentence(s) you kept in Step 1 against that abstract and produce a real verdict: if the paper uses the source as though it were about a different topic than the abstract shows, that is a MISQUOTE; otherwise it is consistent. Writing "-" in the Misquote column for a found row that has an abstract is an ERROR. Only when there is no abstract, or the reference is never cited in the body, is the verdict "unclear". Never invent a verdict.

STEP 4 - PRESENT (follow this format exactly). The user must NEVER see the raw JSON from the function - it is a machine interface. Convert it into ONE readable Markdown table, one row per reference, sorted by DESCENDING severity. Use the "severity" value; if you judge a reference to be a misquote MISMATCH, raise its severity to AT LEAST 80 (keep the function's number if it is already higher). Columns:

  | # | Flag | Reference (as printed) | Status | Metadata issues | Misquote | What the paper is about |

- Flag: blue if status is lookup_failed (could not be checked); otherwise red if

```
  severity >= 70, amber if 45-69, green otherwise.
- Status: the "badge" value with a coloured dot - red Potential hallucination
  (not_found), amber Fuzzy match (fuzzy), blue Lookup failed (no result to
  compare), green Verified (found).
- Metadata issues: if "mismatched_fields" is empty, write "-". Otherwise, for each
  mismatched field take reference_value and openalex_value from "field_check" and
  write:  field: "printed" -> "OpenAlex".  Bold **authors** when it is one of them.
  (A fuzzy row has no field_check yet - write "-".) Fields with status "close"
  ("minor_fields") are NOT errors - leave them out of this column, or mention
  them only as "minor naming variant" without a flag.
- Misquote: Mismatch (red) / Likely mismatch (amber) / Consistent (green) /
  Unclear (dash), plus a 6-10 word reason. (Only "found" rows have a misquote.)
- What the paper is about: one short clause from the "work" abstract, or "-".

After the table, add a "Details" section for the red and amber rows only.
- FOUND row: the matched title as a link to "work.url", the field-by-field
  comparison, and one sentence on the misquote if any.
- FUZZY row: "work" is null - instead list "candidates" (each title as a link to
  its "url", with authors and year) and ask the user to confirm which, if any, is
  the intended source. Do NOT present a fuzzy row as verified.

Then, if Step 1 found orphan citations, add a "Missing references" section: one
line per orphan - the citation as printed, the sentence where it appears, and
the note that it has no entry in the reference list so it cannot be verified.
This is a real problem for the reader and must never be silently omitted.

End with one summary line that does NOT hide problems inside "verified":
"N references - X verified & consistent, Y need review, Z fuzzy, W potential
hallucinations, V in-text citations missing from the reference list." (Drop the
last part when there are no orphans.) A reference that EXISTS but has a metadata
mismatch or a misquote counts under "need review", never under "verified"
(optionally break Y down, e.g. "2 with wrong details, 1 cited out of context").

Never invent a DOI, author, year, or verdict the function did not return. If
"status" is "lookup_failed", say the reference could not be checked - do NOT call
it a hallucination.
```

Step 3 — Add the Header

In the Headers section add: **Key** `Content-Type` **Value** `application/json`.

Optional — OpenAlex Premium: add a second header `X-OpenAlex-Key` with your key as the value, using the Secure header values option so it is stored in Azure Key Vault.

Step 4 — Add the function

Method & URL

```
POST  https://YOUR-DEPLOYMENT-HOST/api/verify_batch
```

If POST sends an empty body on your platform (see Troubleshooting), use GET with the same URL instead — the endpoint accepts the arguments in the query string too.

Function definition

```
{
  "name": "verify_references",
  "description": "Verify a list of bibliographic references against OpenAlex.
    Call this whenever the user says 'start', 'start the extension', 'start
    the check', or wants to check, verify, or fact-check the citations or
    references in a paper. For each reference, returns whether the work exists
    (found / fuzzy / not_found), a field-by-field comparison of the printed
    metadata (title, authors, year, journal, DOI, volume, issue, pages) against
    the real record, and the abstract of the matched work.",
  "parameters": {
    "type": "object",
    "properties": {
      "references": {
        "type": "array",
        "description": "Every reference in the paper's bibliography.",
        "items": {
          "type": "object",
          "properties": {
            "title":   { "type": "string" },
            "authors": { "type": "array", "items": { "type": "string" },
                         "description": "Every author name printed, in order." },
            "et_al":   { "type": "boolean" },
            "year":    { "type": "integer" },
            "doi":     { "type": "string" },
            "journal": { "type": "string" },
            "volume":  { "type": "string" },
            "issue":   { "type": "string" },
            "pages":   { "type": "string" }
          },
          "required": ["title"]
        }
      }
    },
    "required": ["references"]
  }
}
```

Step 5 — Submit and test

Click Submit. In a chat, upload a paper and type “start” (or “start the extension”, “check my citations”, …) — the Detail description tells the assistant to treat any such go-ahead as the run signal and call your endpoint without asking questions first. If a short trigger gets a menu instead of a run, use the most dependable, explicit prompt: “Use the execution steps in the verify_references extension in order to check the references in the attached pdf.”

Installed this extension before? The saved copy does not update itself: if “start the extension” gets you a menu instead of a run, re-paste the current Detail description from Step 2 into your extension and submit again.

Troubleshooting — count: 0

Every response carries an api_version field, and a response with zero results carries a hint field describing what the server received (key names only, never content). Ask the assistant to show the raw output: no api_version means the extension calls an OLD deployment — fix the function URL (preview/branch URLs keep serving stale builds). A hint saying the body was EMPTY (Content-Length: 0) means the platform never put the function arguments into the POST body — change the function's Method from POST to GET (arguments then travel in the query string, which the endpoint accepts), or fill in the builder's request-body/template field if it has one. Any other hint describes the shape that arrived — send it to the maintainer. You can test the endpoint from a terminal: curl -s -X POST /api/verify_batch -H "Content-Type: application/json" -d '{"references":[{"title":"Attention is all you need","year":2017}]}' — a healthy deployment answers within seconds with count: 1 and a Verified result.

What the endpoint returns

Request EduGenAI sends to /api/verify_batch:


```
{
  "references": [
    {
      "title": "Highly accurate protein structure prediction with AlphaFold",
      "authors": ["Smith, J.", "Jones, B."],
      "et_al": false,
      "year": 2019,
      "journal": "Science",
      "pages": "100-110"
    }
  ]
}
```


Response (abbreviated) — wrong authors, year and journal are caught, and the abstract is returned for the misquote step:


```
{
  "count": 1,
  "results": [{
    "index": 1,
    "status": "found",
    "badge": "Verified",
    "severity": 85,
    "priority": "Review",
    "mismatched_fields": ["authors", "year", "journal"],
    "minor_fields": [],
    "work": {
      "title": "Highly accurate protein structure prediction with AlphaFold",
      "authors": ["John Jumper", "Richard Evans", "..."],
      "year": 2021, "venue": "Nature",
      "doi": "10.1038/s41586-021-03819-2",
      "abstract": "Proteins are essential to life ... (used for the misquote check)"
    },
    "field_mismatch_count": 3,
    "field_check": [
      { "field": "title",   "status": "match" },
      { "field": "authors", "status": "mismatch",
        "reference_value": "Smith, J., Jones, B.",
        "openalex_value": "John Jumper, Richard Evans, ..." },
      { "field": "year",    "status": "mismatch",
        "reference_value": 2019, "openalex_value": 2021 },
      { "field": "journal", "status": "mismatch",
        "reference_value": "Science", "openalex_value": "Nature" }
    ]
  }]
}
```


A single-reference endpoint is also available at `POST /api/verify` (same fields, no outer “references” array). Prefer the batch endpoint for a whole bibliography — one round trip, kinder to OpenAlex rate limits.

From JSON to a readable table — what you'll see in chat

You never look at that JSON — it is the machine interface between EduGenAI and the endpoint. The Detail description from Step 2 instructs the assistant to convert it (STEP 4) into a severity-sorted table with flags, scores, and the mismatches spelled out, mirroring how the website presents results. A typical chat answer looks like:

```
| # | Flag | Reference (as printed)      | Status              | Metadata issues                | Misquote
|---|------|-----------------------------|---------------------|--------------------------------|----------
| 3 | RED  | Zorblatt, Q. (2021). Quantum blockchain effects on medieval cheese trading... | Potential halluc
| 2 | RED  | Smith, J. & Jones, B. (2019). Highly accurate protein structure prediction... | Verified | autho
| 1 | GREEN | Vaswani, A. et al. (2017). Attention is all you need. NeurIPS. | Verified | - | Consistent | At
```

...followed by a Details section for the flagged rows and a one-line summary count. If you still see raw JSON in the chat, the Detail description probably wasn't saved with the extension (re-check Step 2) or was truncated — re-paste it, or as a per-chat override end your request with: “Present the results as a

severity-sorted table with flags and the metadata mismatches spelled out; do not show raw JSON." The endpoint's JSON ships ready-made display fields (badge, severity, priority, mismatched_fields) precisely so the assistant only has to lay them out, not compute them.

What data passes through your server?

Because the extension calls your deployment, here is exactly what travels there — and what does not:

- **The paper's full text, the student's name, and the citing sentences stay inside EduGenAI.** The assistant keeps the citing sentences for its own misquote reasoning; they are never sent to your endpoint.
- Only reference **metadata** — title, authors, year, DOI, journal, volume, issue, pages — reaches your endpoint, and (titles/DOIs) go on to OpenAlex for the lookup.
- No LLM key is involved server-side; the reasoning runs on EduGenAI's model.

So: only bibliographic metadata goes through your server, and only in transit. The function call is the only thing that reaches it. This app is stateless — no database, no file writes — and never logs request bodies or keys. Whatever hosting stack or reverse proxy you deploy behind may keep its own access log (method + path + status, not bodies); that part is under your control.

Caveat about the OpenAlex Premium key: it is used per-request, never stored, and scrubbed from any error message this app returns. But when your endpoint calls OpenAlex the key rides as a query parameter on the outbound URL, so an egress proxy that logs full outbound URLs could capture it. If that matters, omit the Premium key (the tool works without one) or use a proxy that redacts query strings.

Notes & limits

- No LLM key is stored or used by the extension; OpenAlex is free and needs no key.
- OpenAlex's canonical year can differ from a printed year (online-first vs issue year) — treat a lone year mismatch as a prompt to double-check, not a verdict.
- The misquote judgement uses the abstract only, so a claim supported by the full text but not the abstract may read as uncertain. Treat results as leads for a human reviewer.
- Batch requests are capped at 200 references.